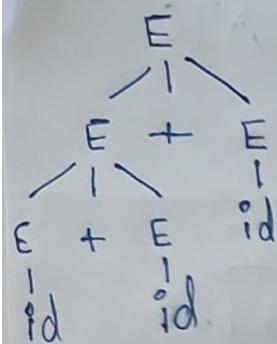
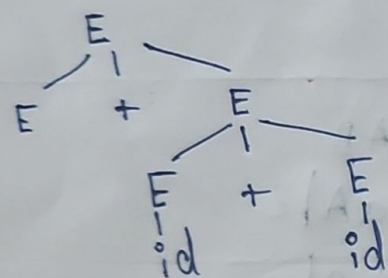


Q1) $E \rightarrow E + E \mid id$
 $(id + id) + id$



$(id + id) + id$ — ①

$id + (id + id)$



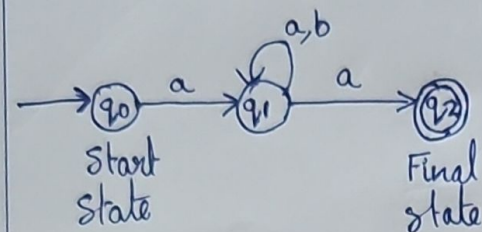
$id + (id + id)$ — ②

① = ②

Ambiguous

Q2) NFA for $a(a|b)^*a$

→ should start with a
 → Followed by a's & b's
 → Ends with a



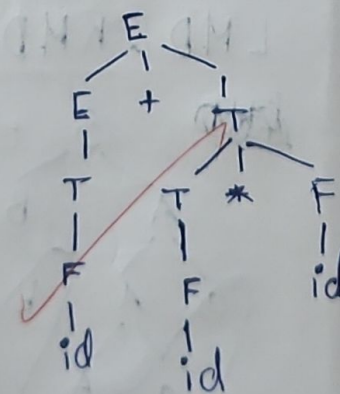
Accepted strings

aabaaba
 aabbba
 aba
 aaa
 abba

Q3) $id + id * id$
 Use a suitable grammar.

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow id$

$E \rightarrow E + T$
 $T \rightarrow T * F$
 $F \rightarrow id$
 $id + T$
 $id + T * F$
 $id + F * F$
 $id + id * id$



①

Q5) $S \rightarrow iE + S \mid iE + S \mid a$

$S \rightarrow iE + S$

$S' \rightarrow eS \mid E$

Left factoring

$S \rightarrow iE + S$

$S \rightarrow iE + S \mid a$

$S \rightarrow a$

$S \rightarrow iE + S$

$S' \rightarrow eS \mid E$

100

Q4) $A \rightarrow A\alpha \mid B$

$A \rightarrow A\alpha$

$A \rightarrow B$

$A' \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$

Q7) $S \rightarrow SaSb \mid cd$

$A \rightarrow A\alpha$

$S \rightarrow Sa$ ✓

$S \rightarrow Sb$ ✓

$S \rightarrow cd$ ✓

∴ The grammar contains immediate left recursion.

②

M. Vishnu prasad
 192311217

Q8) $E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$FIRST(E') = \{+, \epsilon\}$

Q9) $S \rightarrow (s)S \mid \epsilon$

$S \rightarrow (s)S$

$((s)s)S$

$((\epsilon)\epsilon)\epsilon$

⇓

$((\epsilon))$

∴ It is a valid string

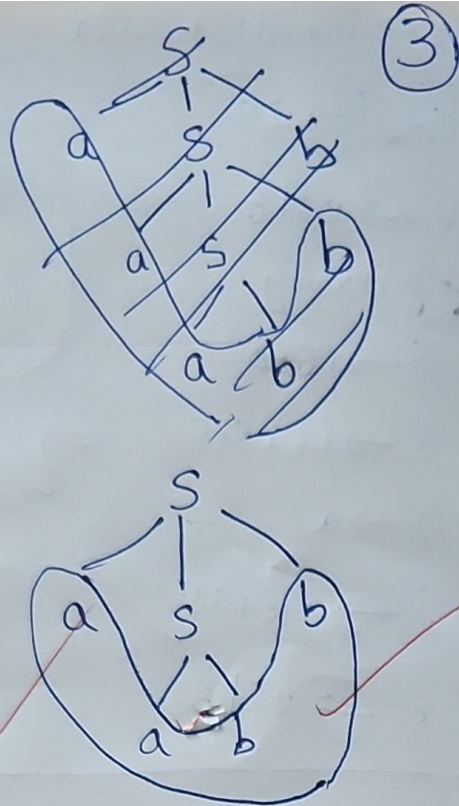
Q10) Shift:- Current symbol from I/P buffer to stack.

Reduce:- Pops out the RHS and pushes the LHS of the production.

Accept:- Includes \$ symbol, the I/P string is accepted.

Error:- If parser cannot perform shift or reduce.

Q11)



∴ aabb is accepted.

Q12) $S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

$\text{First}(S) = \{a, b\}$

$\text{First}(A) = \{a\}$

$\text{First}(B) = \{b\}$

$S \rightarrow asblab$
 $aabb$

Q13) $S \rightarrow A$

$A \rightarrow aB/Ad$

$B \rightarrow b$

$C \rightarrow g$ First & Follow

$S \rightarrow A$

$A \rightarrow aBA'$ $\text{First}(S) = \{a\}$

$A' \rightarrow dA'$ $\text{First}(A) = \{a\}$

$B \rightarrow b$ $\text{First}(A') = \{d\}$

$C \rightarrow g$ $\text{First}(B) = \{b\}$

$\text{First}(C) = \{g\}$

$\text{Follow}(S) = \{\$ \}$

$\text{Follow}(A) = \{\$ \}$

$\text{Follow}(A') = \{\$ \}$

$\text{Follow}(B) = \{d, \$ \}$

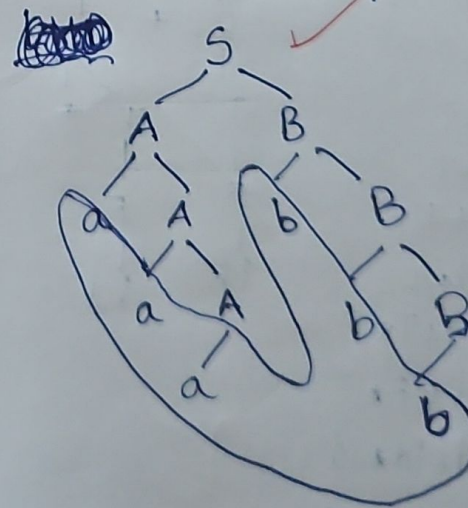
$\text{Follow}(C) = \{d, \$ \}$

Q15) $S \rightarrow AB$

$A \rightarrow aA/a$

$B \rightarrow bB/b$

LMD & RMD for "aaaabbb"



Q14) Len code:-

% {

#include <stdio.h>

% }

% .

"+"

;

% .

int yycwrap() { return 1; }

int main()

{ yylen(); }

Q17) Left Recursion:-

$A \rightarrow A\alpha / B$

$A \rightarrow \beta A'$

$A' \rightarrow aA' / \epsilon$

Left Factoring:-

$A \rightarrow \alpha\beta / \alpha\beta$

$A \rightarrow \alpha A'$

$A' \rightarrow \beta / \beta$

Q16) $S \rightarrow AB$

$A \rightarrow aA / \epsilon$

$B \rightarrow bB / c$

compute follow.

$\text{First}(S) = \{a, b, c, \epsilon\}$

$\text{First}(A) = \{a, \epsilon\}$

$\text{First}(B) = \{b, c\}$

$\text{Follow}(S) = \{\$ \}$

$\text{Follow}(A) = \{b, c, \$ \}$

$\text{Follow}(B) = \{b, c, \$ \}$

Q19) $E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

$E \rightarrow id$

$id_1 + id_2 * id_3$

(b)

Stack

SIP buffer

Operation.

\$

id + id * id \$

Shift

\$id

+ id * id \$

Reduce $E \rightarrow id$

\$E

+ id * id \$

Shift

\$E+

id * id \$

Shift

\$E+id

* id \$

Reduce $E \rightarrow E + E$

\$E+E

* id \$

Reduce $E \rightarrow E + E$

\$E

* id \$

Shift

\$E*

id \$

Shift

\$E*id

\$

Reduce $E \rightarrow id$

\$E*E

\$

Reduce $E * E \rightarrow E$

\$E

\$

Accept.

20) #include <stdio.h>

```
int main() {  
    int a = 10, b = 20;  
    int sum = a + b;  
    printf("%d", sum);  
    return 0;  
}
```

a) C codes process through phases of the compiler.

→ Lexical:-

→ Syntax:-

→ Semantic:-

Intermediate
Code generator:-

t1 = a + b

sum = t1

t2 = sum

t3 = print(sum)

Code optimization:-

t1 = sum(a + b)

t2 = print(sum).

(7)

code generator:-

LOAD R1

LOAD R2

ADD R1, R2

MOV R1, #10

MOV R2, #20

ADD R3, R1, R2

MOV sum, R3

call printf

X

(8)